A

Mini Project Report

On

Dynamic Data Visualization Dashboard using Python

Submitted to DR.K.V.SUBBA REDDY INSTITUTE OF TECHNOLOGY

In Partial Fulfillment of the requirements for the Award of Degree of

BACHELOR OF
TECHNOLOGY IN
COMPUTER SCIENCE AND ENGINEERING(AIML/AI)

Submitted By

Shaik Hamza Mohammed

_____

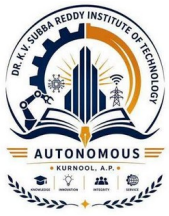
_____

_____

Under the Esteemed guidance of

Dr. B.MAHESH

Associate Professor, Department of CSE



Department of CSE(AI & AIML)

DR.K.V.SUBBA REDDY INSTITUTE OF TECHNOLOGY

UGC AUTONOMOUS

(Approved by AICTE, NEW DELHI, Affiliated to JNTUA, Anantapur)

2026-2027

DR.K.V.SUBBA REDDY INSTITUTE OF TECHNOLOGY

UGC AUTONOMOUS

(Accredited by NBA,Approved by AICTE NEW DELHI, Affiliated to JNTUA, Anantapur)

Department of CSE(AI/AIML)

CERTIFICATE

This is to certify that the project entitled **"Dynamic Data Visualization Dashboard using Python"** is a bonafide work carried out by

Shaik Hamza Mohammed

in partial fulfillment of the requirement for the award of the degree of **BACHELOR OF TECHNOLOGY** in **CSE(AI/AIML)** from Dr.K.V.Subba Reddy Institute of Technology, Kurnool, under our guidance and supervision.

The results presented in this Mini project have been verified and are found to be satisfactory. The results embodied in this Mini project have not been submitted to any other university for the award of any other degree or diploma.

Guide **Head of the Department**

DECLARATION

This is to certify that the work reported in the present Mini project entitled ‘‘Dynamic Data Visualization Dashboard using Python” is a record of bonafide work done by us in the Department of CSE(AI/AIML), Dr.K.V.Subba Reddy Institute of Technology, Kurnool. The reports are based on the project work done entirely by us and not copied from any other source. We submit our project for further development by any interested students who share similar interests to improve the project in the future.

The results embodied in this Mini project report have not been submitted to any other University or Institute for the award of any degree or diploma to the best of our knowledge and belief.

ACKNOWLEDGMENT

We are extremely grateful to Dr. K Sree Latha, Principal and H. Ateeq Ahmed, HOD, Department of CSE(AI/AIML), Dr.K.V.Subba Reddy Institute of Technology for their constant support. We are extremely thankful to Dr. B.Mahesh, Associate Professor, Department of CSE, Internal Guide, for his constant guidance, encouragement and moral support throughout the project.

We will be failing in duty if We do not acknowledge with grateful thanks to the authors of the references and other literatures referred in this Project.

We express our thanks to all staff members and friends for all the help and coordination extended in bringing out this project successfully in time.

Finally, we are very much thankful to our parents who guided us for every step.

TABLE OF CONTENTS

1. ABSTRACT 7

2. INTRODUCTION 7

2.1 Background 7

2.2 Motivation 7

2.3 Problem Statement 8

2.4 Objectives of the Project 8

2.5 Scope of the Project 8

3. LITERATURE SURVEY / EXISTING SYSTEM 8

3.1 Existing Data Visualization Tools 8

3.2 Limitations of Existing System 8

3.3 Proposed System 8

4. TECHNOLOGIES AND TOOLS USED 9

4.1 Python 9

4.2 Streamlit 9

4.3 Pandas 9

4.4 NumPy 9

4.5 Plotly 9

4.6 Streamlit Community Cloud 9

5. SYSTEM REQUIREMENTS 9

5.1 Hardware Requirements 9

5.2 Software Requirements 9

6. DATASET DESCRIPTION 9

6.1 Dataset Sources 10

6.2 Dataset Attributes 10

6.3 Data Types 10

6.4 Data Preprocessing 11

6.5 Data Cleaning 11

6.6 Summary Statistics 11

7. IMPLEMENTATION 12

7.1 Project Structure 12

7.2 Data Loading 12

7.3 Data Preprocessing 12

7.4 Visualization Development 12

7.5 Streamlit Interface . . . 12
7.6 Interactive Filters . . . 13
7.7 Dashboard Integration . . . 13
7.8 Deployment . . . 13

8. DATA VISUALIZATION . . . 13

8.1 Overview Charts . . . 13
8.2 Subject Analysis Charts . . . 13
8.3 Student Profile and Comparison Charts . . . 13
8.4 Correlation and Scatter Charts . . . 14

9. RESULTS AND DISCUSSION . . . 14

9.1 Dashboard Figures . . . 14
9.2 Visualization Results . . . 21
9.3 Analysis of Results . . . 22
9.4 Key Insights . . . 23

10. ADVANTAGES AND LIMITATIONS . . . 23

10.1 Advantages . . . 23
10.2 Limitations . . . 24

11. CONCLUSION . . . 24

12. FUTURE ENHANCEMENT . . . 24

13. REFERENCES . . . 24

14. APPENDIX . . . 25

A. Source Code . . . 25
B. Dataset Notes . . . 33
C. Live Deployment . . . 33

1. ABSTRACT

The Student Marks Analytics Dashboard is a data-visualization project that explores the academic performance of 500 students across six subjects — Mathematics, Physics, Chemistry, English, Computer Science and Biology. The dashboard brings the marks, attendance and study-habit data of these students into one interactive web interface so that users can filter students, compare subjects, sections and individual students, and inspect trends and relationships. Python and Streamlit provide the application, Pandas and NumPy prepare and summarise the data, and Plotly produces the interactive charts.

The dashboard has six pages — Overview, Subject Analysis, Student Profile, Compare, Correlations and Data Explorer — together with a sidebar of filters, summary (KPI) cards and a filtered-data download. The dataset used in this project is synthetic: it was generated with a Python script for demonstration purposes and does not contain records of real students.

Keywords: Data visualization, student performance, marks analysis, dashboard, Python, Streamlit, Plotly, Pandas.

2. INTRODUCTION

2.1 Background

Educational institutions record large amounts of data about their students: marks in each subject, attendance, and sometimes study habits. These records are usually stored in tables or spreadsheets. A table of several hundred students is difficult to read at a glance, and questions such as *which subject is hardest*, *how are grades distributed*, *do sections differ* or *does attendance relate to marks* are hard to answer by looking at rows of numbers.

Data visualization. Data visualization is the practice of representing data with visual elements such as bars, lines, points and colours so that patterns, comparisons and unusual values can be seen quickly. Different chart types suit different questions: bar charts compare categories, histograms and box plots show how values are spread, radar charts compare several measures of one student or group at once, heatmaps show many relationships in one picture, and scatter plots show the relationship between two numeric variables.

Dashboards and interactivity. A dashboard collects related summary figures and charts on one screen. Interactive controls such as filters, sliders and drop-down lists let the user change what is displayed instead of reading a fixed report. In a Python dashboard built with Streamlit, the script is re-run whenever a control changes, so the numbers and charts are recalculated from the selected data. Caching is used so that the data file is not reloaded on every interaction.

The Python data stack. Pandas provides the DataFrame structure used to load, clean, filter and aggregate tabular data; NumPy supports numerical operations; and Plotly produces interactive charts with hover details, zooming and panning. Together with Streamlit, they allow a complete data application to be written in a single Python file and run in a web browser.

2.2 Motivation

Teachers and students often want a quick, clear picture of academic performance: where the class is strong, where it is weak, and which students may need support. The Student Marks Analytics Dashboard aims to make this information easy to explore through interactive charts instead of requiring users to compare many separate tables. The project is also a practical exercise in the full data-visualization workflow: preparing a dataset, cleaning and validating it, choosing suitable charts, and presenting the result in a deployed interactive application.

2.3 Problem Statement

Student performance data stored in plain tables is large, hard to compare by eye, and cannot be filtered or re-grouped without extra work. Static charts answer only the question they were drawn for. The project addresses this by organising student marks, attendance and study hours into one dashboard that provides summary figures, interactive charts and filters, and that calculates results such as percentage, grade and pass or fail automatically.

2.4 Objectives of the Project

- Load and validate a student marks dataset and calculate total marks, percentage, result and grade for every student.
- Summarise overall performance with KPI cards: number of students, average, highest and lowest percentage, pass rate and average attendance.
- Compare subjects, sections, genders and grade groups with suitable Plotly charts.
- Show the profile of any individual student against the class average and compare several students side by side.
- Explore relationships between subjects, attendance, study hours and overall percentage using a correlation heatmap and a scatter explorer.
- Provide filters, search, sorting and a download of the filtered data, and deploy the dashboard online.

2.5 Scope of the Project

The project is an educational data-visualization dashboard. Within its scope, the dashboard covers 500 students in four sections (A–D), six subjects marked out of 100, and two study-habit measures (attendance and daily study hours), presented on six pages. The following are outside the scope of the project: predicting the future marks or results of any individual student, working with real student records and their privacy requirements, proving that one factor (such as study time) causes higher marks, and user login or role-based access.

3. LITERATURE SURVEY / EXISTING SYSTEM

3.1 Existing Data Visualization Tools

Spreadsheets can sort student marks and draw basic charts. Business-intelligence platforms offer interactive dashboards, learning-management systems provide built-in grade reports, and Python libraries allow customised analysis inside a code-based workflow.

3.2 Limitations of Existing System

- Spreadsheet charts must be rebuilt by hand whenever the group of students to be analysed changes.
- Learning-management reports usually show fixed views and offer little freedom to combine subjects, sections and study habits.
- Commercial dashboard products may be unnecessary or costly for a small academic project.
- Plain tables of marks make comparison between subjects, sections and students slow and error-prone.

3.3 Proposed System

The proposed system is a Streamlit-based dashboard that presents student marks with interactive filters and visualizations. It recalculates every figure from the selected students, offers six task-oriented pages, and can be run locally with one command or opened online after deployment.

4. TECHNOLOGIES AND TOOLS USED

4.1 Python

Python is used for the application logic, data validation and calculations.

4.2 Streamlit

Streamlit provides the web interface: the sidebar, KPI cards, tabs, drop-down lists, sliders, data tables and the download button. Its caching (`st.cache_data`) is used to load the dataset only once.

4.3 Pandas

Pandas loads the CSV file, checks its columns, converts data types, removes duplicates and computes group-wise summaries such as averages per subject, section or gender.

4.4 NumPy

NumPy is used for numerical operations such as pass/fail tests on whole columns, rounding of the axis limits for the sliders, and the least-squares trend line (`numpy.polyfit`) in the scatter explorer.

4.5 Plotly

Plotly (`plotly.express` and `plotly.graph_objects`) draws all interactive charts — bar, histogram, donut, box plot, radar, heatmap and scatter — with hover details, zoom and pan.

4.6 Streamlit Community Cloud

The finished application is deployed on Streamlit Community Cloud so that it can be opened from a web browser without installing Python.

5. SYSTEM REQUIREMENTS

5.1 Hardware Requirements

- Laptop or desktop computer with a modern processor.
- 4 GB RAM or more recommended.
- A few hundred megabytes of storage for Python, the libraries and the project files (the dataset itself is only about 37 KB).
- Internet access to install the libraries and to open the deployed dashboard.

5.2 Software Requirements

- Python 3.10 or later and pip.
- Streamlit (1.40 or later), Pandas (2.0 or later), NumPy (1.24 or later) and Plotly (5.20 or later), as listed in requirements.txt.
- A code editor such as Visual Studio Code and a modern web browser.

Typical launch command, from the project folder: `python -m streamlit run app.py`

6. DATASET DESCRIPTION

The dashboard works with one data file, `student_marks_dataset.csv`, stored in the `data` folder. The table below gives its size at a glance; the sub-sections that follow explain the source, every attribute (column), the data types and the preparation steps in simple terms.

File	What it contains	Size (rows)	Columns	Used on
student_marks_dataset.csv	Marks in six subjects, attendance and study hours of students, with total, percentage, result and grade	500 students	18	All six pages

6.1 Dataset Sources

The dataset is **synthetic**. It was created for this project with a Python script (NumPy random generator and Pandas) so that the dashboard can be demonstrated without using any real student's records. For each student, an overall ability level was drawn at random and combined with the student's attendance and daily study hours to give a base level of performance; each subject then received a small subject-specific adjustment (Mathematics and Physics are slightly harder, English slightly easier), individual variation and random noise, and every mark was limited to the range 0–100. A few unusually low marks were added deliberately so that the charts contain outliers. Total marks, percentage, result and grade were then calculated from the six subject marks.

Because the data are synthetic, the relationships found in the analysis (for example between study hours and marks) reflect the assumptions used to generate the data, not findings about real students.

6.2 Dataset Attributes

Each row is one student and every column is one attribute. The dataset has 18 columns, explained below.

Attribute (column)	Meaning in simple words	Type	Example
Student_ID	Unique ID of the student	Text	STU1001
Student_Name	Name of the student	Text	Harsha Prasad
Gender	Male or Female	Text	Male
Section	Class section (A, B, C or D)	Text	C
Attendance_%	Percentage of classes attended	Number	91.6
Study_Hours_Per_Day	Average hours of study per day	Number	2.1
Mathematics, Physics, Chemistry, English, Computer Science, Biology	Marks obtained out of 100 in each subject (six columns)	Number (whole)	90, 70, 79, 93, 100, 75
Total_Marks	Sum of the six subject marks (out of 600)	Number	507
Percentage	Total marks divided by 600, times 100	Number	84.50
Subjects_Failed	Number of subjects with marks below the pass mark (35)	Number	0
Result	Pass if no subject is below the pass mark, otherwise Fail	Text	Pass
Grade	A+, A, B+, B, C, D or F (see Section 7.3)	Text	A

6.3 Data Types

The columns fall into two kinds of data. The code converts every subject column to a number when the file is loaded, because a number stored as text cannot be averaged or plotted.

Data type	Columns	How it is used
Text (string)	Student_ID, Student_Name, Gender, Section, Result, Grade	Labels on chart axes, colours and legends, search, and grouping students
Numeric (whole / decimal)	Six subject columns, Attendance_%, Study_Hours_Per_Day, Total_Marks, Percentage, Subjects_Failed	Bar heights, histograms, box plots, radar values, correlations, sliders and KPI cards

6.4 Data Preprocessing

The following preparation steps are performed by the application each time it loads the data:

- Column names are trimmed and checked: Student_ID, Student_Name, Gender and Section are required.
- Subject columns are detected automatically: any numeric column that is not an ID, attribute or calculated column is treated as a subject, so a dataset with different subjects also works.
- Subject marks are converted to numbers and limited to the range 0–100.
- Total_Marks (sum of the subjects) and Percentage (average of the subjects, each out of 100) are calculated for every student.
- Subjects_Failed, Result and Grade are calculated from the pass mark chosen in the sidebar (default 35), so they update when the pass mark is changed.
- The data are cached, so the file is not read again on every click, and filters are applied to a copy so the original table is never changed.

6.5 Data Cleaning

- Values that cannot be converted to numbers are turned into missing values instead of stopping the program.
- Missing text values in ID, name, gender and section are replaced by “Unknown”; students with no subject marks at all are removed.
- Duplicate Student_ID rows are removed, keeping the first.
- If the data file is missing, the dashboard shows a message and a file-upload box; if required columns are missing or fewer than two subject columns exist, it shows an error that names the problem.

Important limitation: the dataset is synthetic and small (500 students). It is suitable for demonstrating the dashboard but not for drawing conclusions about real students.

6.6 Summary Statistics

The table below summarises the six subjects for all 500 students (marks out of 100; pass mark 35).

Subject	Mean	Median	Std. dev.	Min	Max	Pass rate %
Mathematics	61.7	62.0	12.6	10	96	98.6
Physics	61.5	61.0	11.2	15	93	99.6
Chemistry	63.5	63.0	11.2	9	95	99.4
English	69.0	69.0	11.3	23	100	99.8
Computer Science	67.7	68.0	11.5	20	100	99.6
Biology	65.8	65.0	11.2	26	100	99.4

7. IMPLEMENTATION

7.1 Project Structure

The whole dashboard is written as a single Streamlit file, `app.py`, which reads the data file from the `data` folder.

File	Purpose
app.py	Streamlit application: data loading, validation, filters, calculations, charts and page layout
requirements.txt	List of Python libraries to install (streamlit, pandas, numpy, plotly)
README.md	Instructions for running the project and description of the pages
data/student_marks_dataset.csv	Student marks dataset (500 students, 18 columns)

7.2 Data Loading

The function `load_path()` reads the CSV file with Pandas and passes it to `prepare()`. Both are cached with `@st.cache_data`, and the cache is refreshed if the file is modified. If the file is not found, `get_dataset()` shows a warning and offers a file uploader; errors in the file are reported with a friendly message instead of a crash.

7.3 Data Preprocessing

`prepare()` validates the columns, detects the subject columns, cleans values and adds Total_Marks and Percentage. `derive()` then adds Subjects_Failed, Result and Grade for the pass mark selected in the sidebar. The grading scheme is:

Grade	Condition
A+	Percentage 90 and above, no subject below the pass mark
A	Percentage 80 to 89.99
B+	Percentage 70 to 79.99
B	Percentage 60 to 69.99
C	Percentage 50 to 59.99
D	Percentage below 50, no subject below the pass mark
F	At least one subject below the pass mark (Result = Fail)

7.4 Visualization Development

Charts are chosen according to the question being asked: bar charts for averages and rankings, histograms and box plots for distributions, a donut chart for proportions, radar charts for multi-subject profiles, a heatmap for sections and for correlations, and a scatter plot for relationships. All charts use Plotly with titles, axis labels, hover information and zoom and pan, and a common helper keeps their style consistent.

7.5 Streamlit Interface

Streamlit arranges the page into a sidebar, a row of KPI cards, and one of six pages selected from the sidebar menu:

Page	Content
Overview	Average marks per subject, percentage histogram, grade donut, pass and fail by section, top and bottom 5/10/15 students

Page	Content
Subject Analysis	Tabs: averages and pass rate, box plots, single-subject view, grouped comparison by section/gender/grade/result, section heatmap
Student Profile	One selected student: percentage, rank, grade, result, radar chart and bar chart against the class average
Compare	Radar charts of groups (section, gender, grade, result) or of 2 to 6 chosen students, with tables
Correlations	Correlation heatmap and a scatter explorer (choice of axes and colour) with a trend line
Data Explorer	Search by name or ID, sort, choose number of rows, summary statistics and download of the filtered data

7.6 Interactive Filters

The sidebar contains multi-select filters for Section, Gender and Result (an empty selection means all), a Percentage range slider and an Attendance range slider. An “Advanced controls” panel holds the pass mark slider (0–60, default 35). A caption shows how many of the students match the filters. All KPI cards, charts and tables on every page are recalculated from the filtered students; if no student matches, the dashboard shows a warning instead of empty charts.

7.7 Dashboard Integration

On every run the dashboard follows one workflow: load the cached data → apply the pass mark → apply the sidebar filters → show the KPI cards → draw the selected page → show the footer note. Each page is a separate function, which keeps the code easy to read and extend.

7.8 Deployment

The project was uploaded to a GitHub repository and deployed on Streamlit Community Cloud, which installs the libraries in requirements.txt and runs app.py. The deployed dashboard is available at: https://assignment-2-project-2hs4f5lzywtf2sdwzvk79t.streamlit.app/

8. DATA VISUALIZATION

8.1 Overview Charts

The Overview page uses a vertical bar chart (average marks per subject with the pass mark as a dashed line), a histogram of overall percentage with the mean marked, a donut chart of grade proportions, a grouped bar chart of pass and fail counts by section, and horizontal bar charts of the top and bottom students (5, 10 or 15 selectable).

8.2 Subject Analysis Charts

The Subject Analysis page combines a bar chart of average marks with a line of pass rates, box plots that show the median, spread and outliers of each subject, a histogram of one chosen subject split by gender, grouped bars by section, gender, grade or result, and a section-by-subject heatmap of average marks.

8.3 Student Profile and Comparison Charts

Radar (polar) charts show the six subject marks of one student against the class average, the average profile of each section or other group, or several students at once. A grouped bar chart repeats the single-student comparison in a form that is easier to read exactly.

8.4 Correlation and Scatter Charts

A correlation heatmap shows the relationships among the six subjects, attendance, study hours and overall percentage. The scatter explorer lets the user choose any two of these measures for the axes and colour the points by gender, section, result or grade; a least-squares trend line is added.

9. RESULTS AND DISCUSSION

9.1 Dashboard Figures

The figures below show the charts of the Student Marks Analytics Dashboard for the complete dataset (all 500 students, all sections, pass mark 35). They were generated from the dashboard dataset using the same calculations and chart types as the dashboard, and are arranged page by page as in the sidebar menu. The live dashboard can be opened at the address given in Appendix C.

Overview page

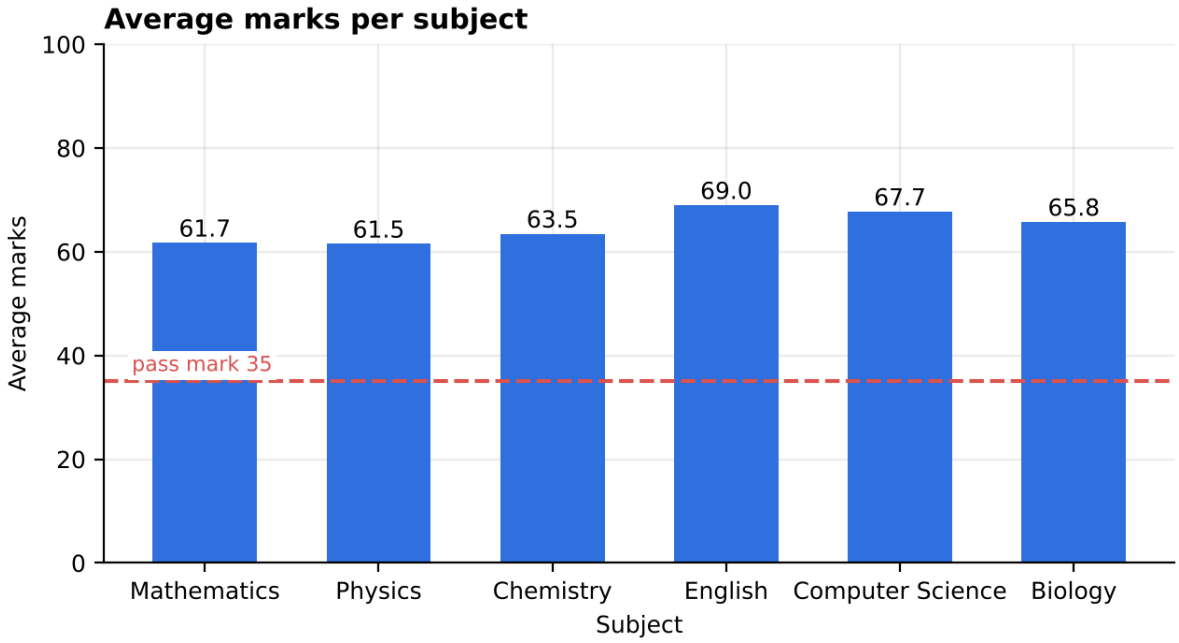


Figure 1: *Overview page: average marks per subject with the pass mark (35) as a dashed line.*

Note: English has the highest average (69.0) and Physics the lowest (61.5). All six averages are far above the pass mark.

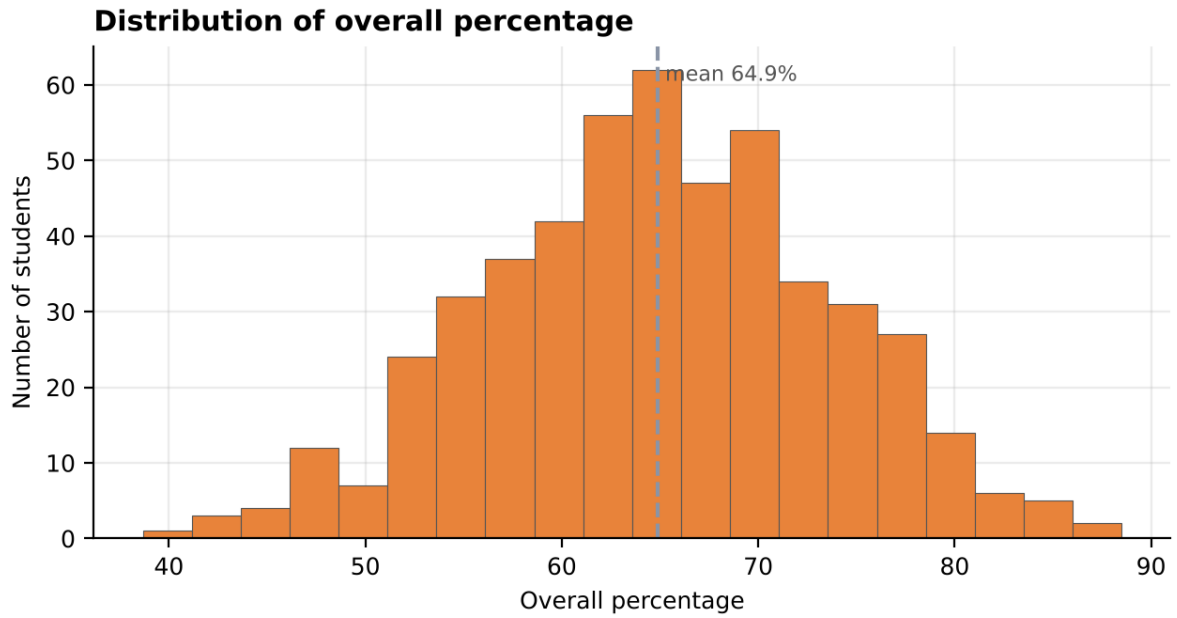


Figure 2: *Overview page: distribution of overall percentage (20 bins).*

Note: The distribution is roughly bell-shaped around the mean of 64.9%. Percentages range from 38.7% to 88.5%.

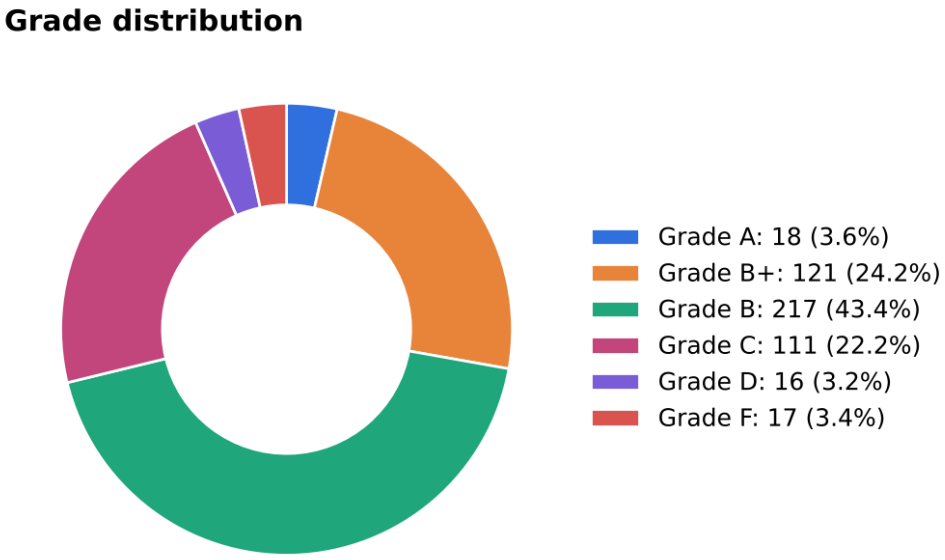


Figure 3: *Overview page: grade distribution.*

Note: Grade B is the largest group (43.4%), followed by B+ (24.2%) and C (22.2%). 17 students (3.4%) receive grade F, and no student reaches A+.

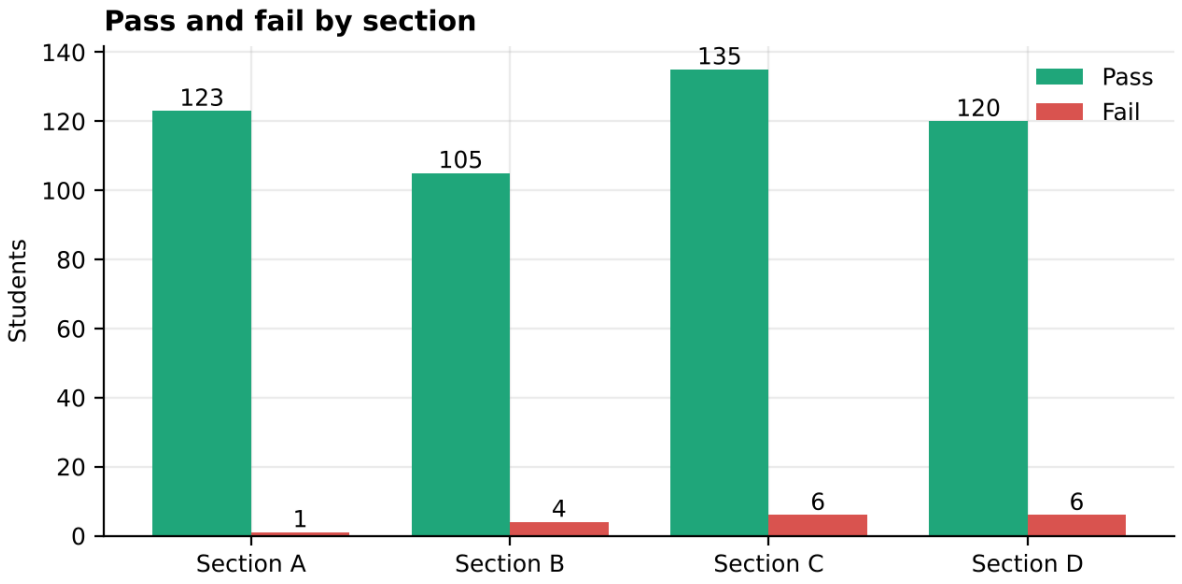


Figure 4: *Overview page: pass and fail counts by section.*

Note: Section C is the largest (141 students). Failures are few in every section: A 1, B 4, C 6, D 6.

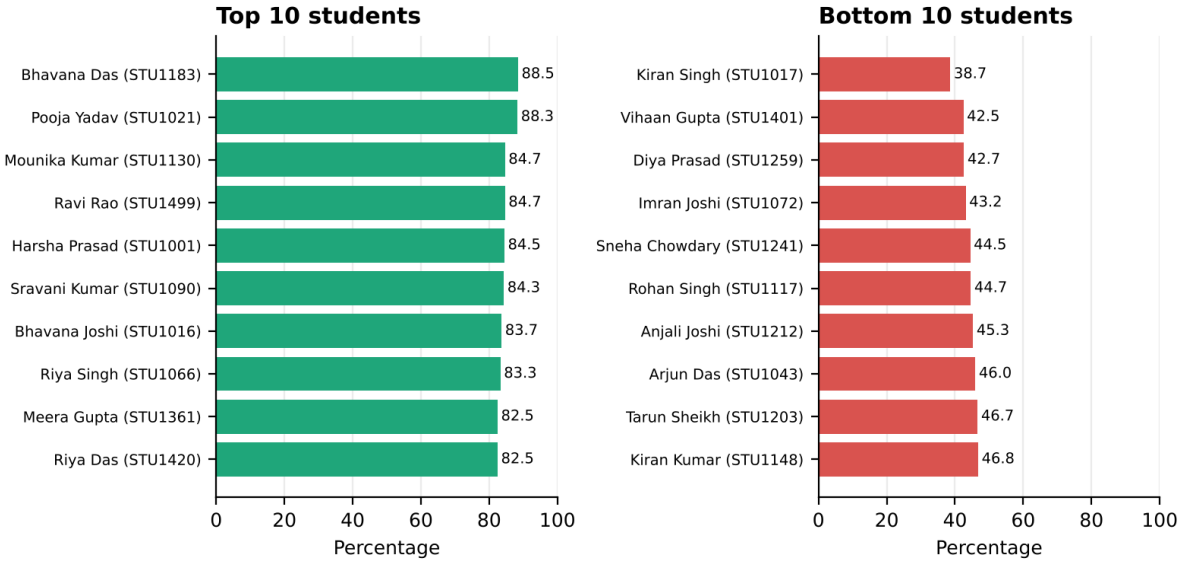


Figure 5: *Overview page: top 10 and bottom 10 students by overall percentage.*

Note: The best result is 88.5% (Bhavana Das, STU1183); the lowest is 38.7% (Kiran Singh, STU1017). Names in the dataset are synthetic.

Subject Analysis page

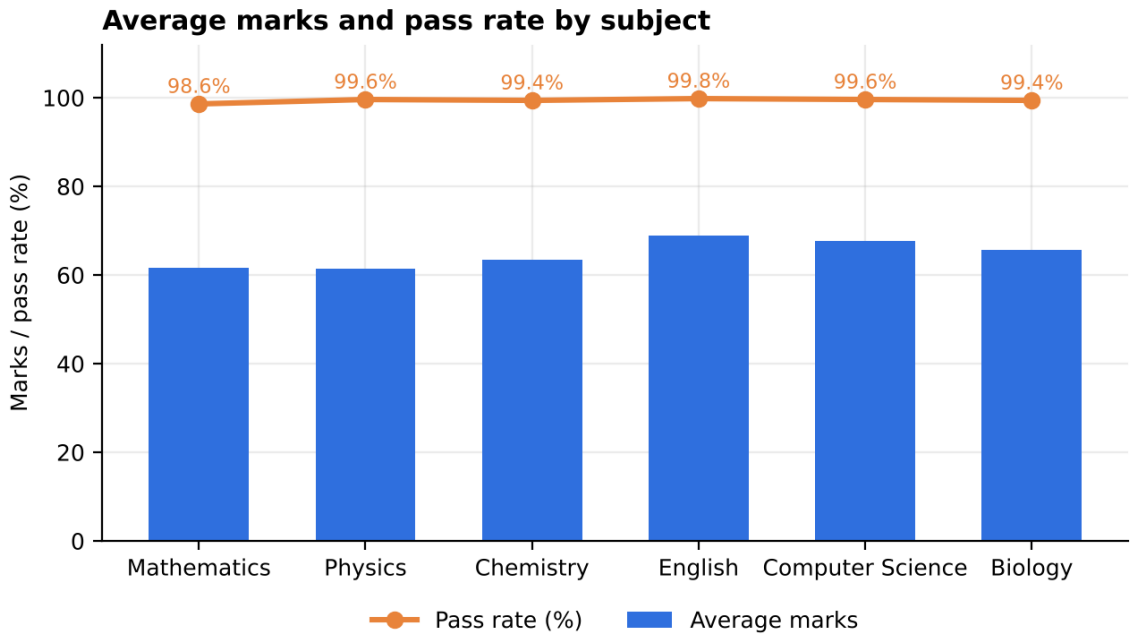


Figure 6: *Subject Analysis page: average marks (bars) and pass rate (line) for each subject.*

Note: Pass rates are between 98.6% and 99.8%. Mathematics has the lowest pass rate (98.6%) and the most marks below 35 (7).

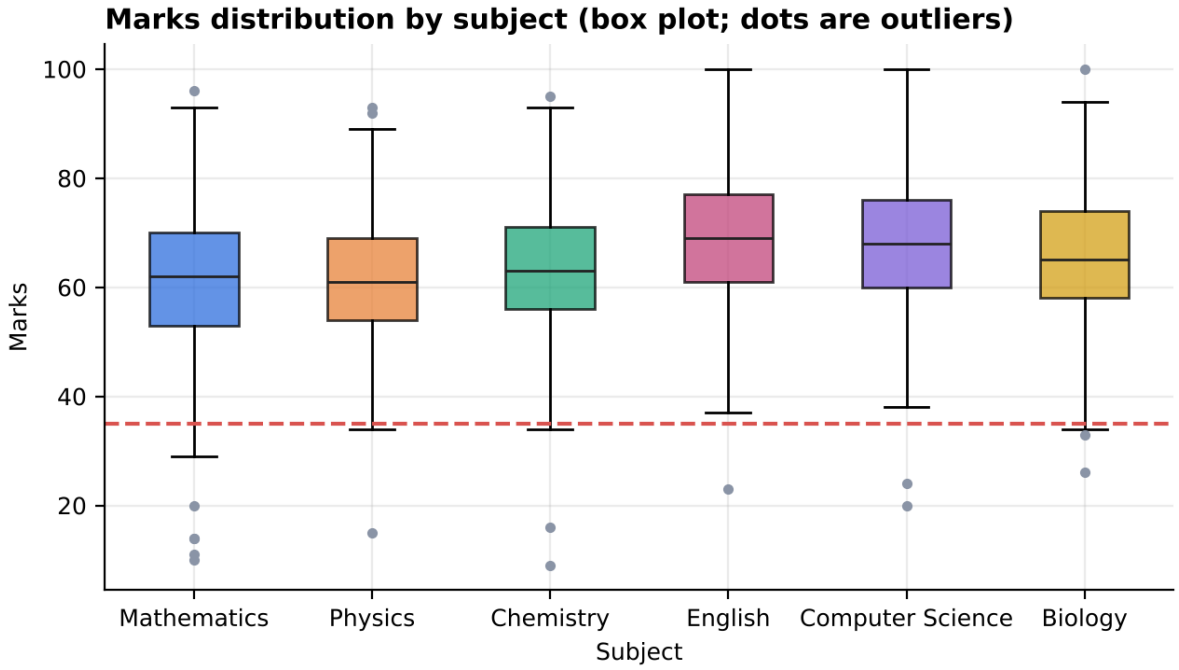


Figure 7: *Subject Analysis page: box plot of marks for each subject; dots are outliers.*

Note: The boxes show the middle 50% of marks and the line inside each box the median. English and Computer Science sit higher than the other subjects; the dots near the bottom are the unusually low marks that were added to the data.

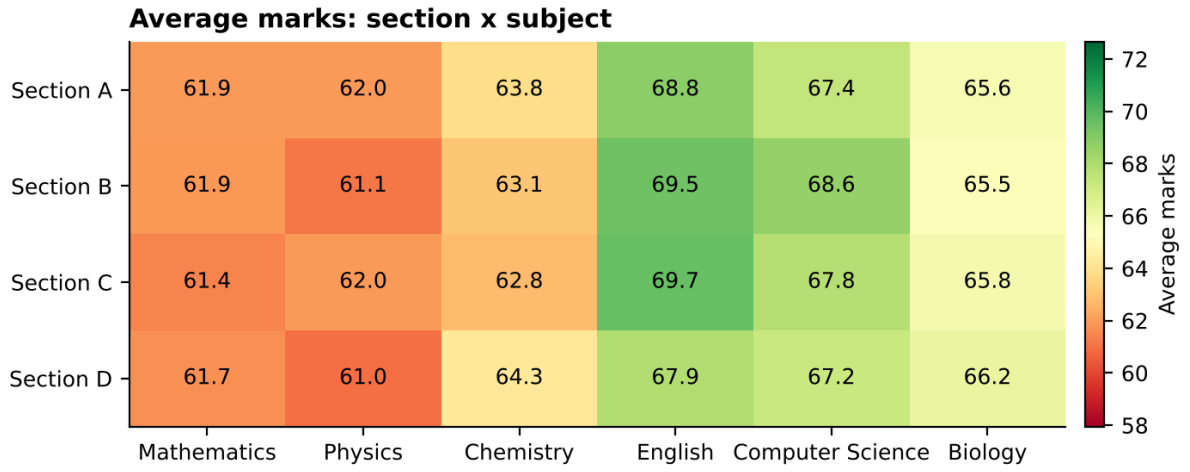


Figure 8: *Subject Analysis page: average marks by section and subject (heatmap).*

Note: Colours differ much more between subjects than between sections: each subject has almost the same average in all four sections.

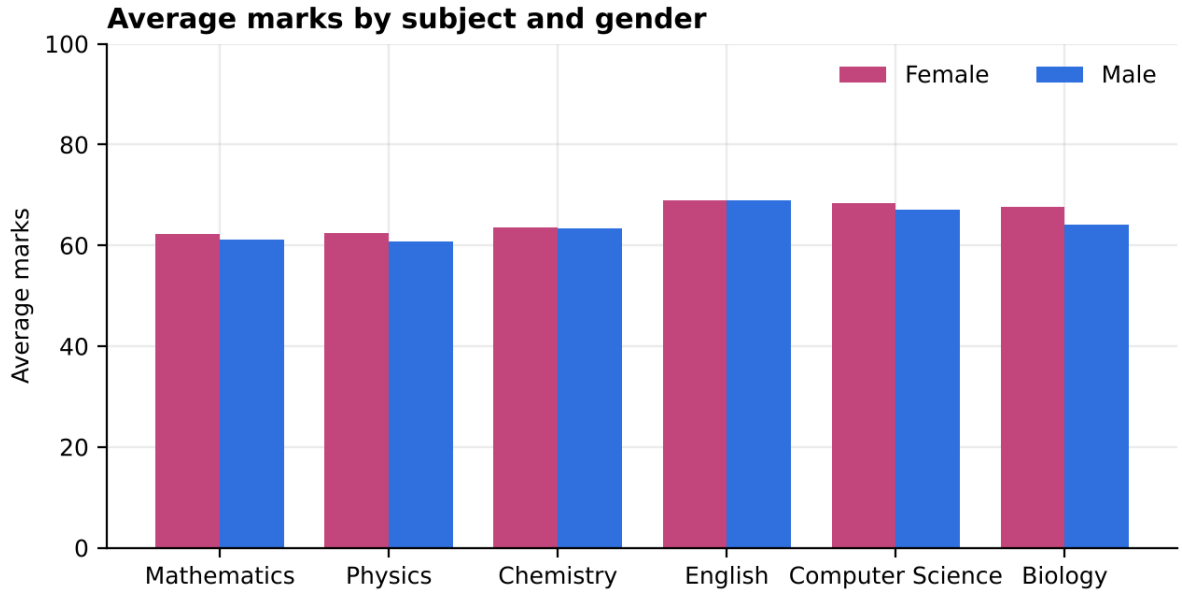


Figure 9: *Subject Analysis page: average marks by subject, grouped by gender.*

Note: Female students average 65.5% overall and male students 64.3%. The gap is small in every subject.

Student Profile and Compare pages

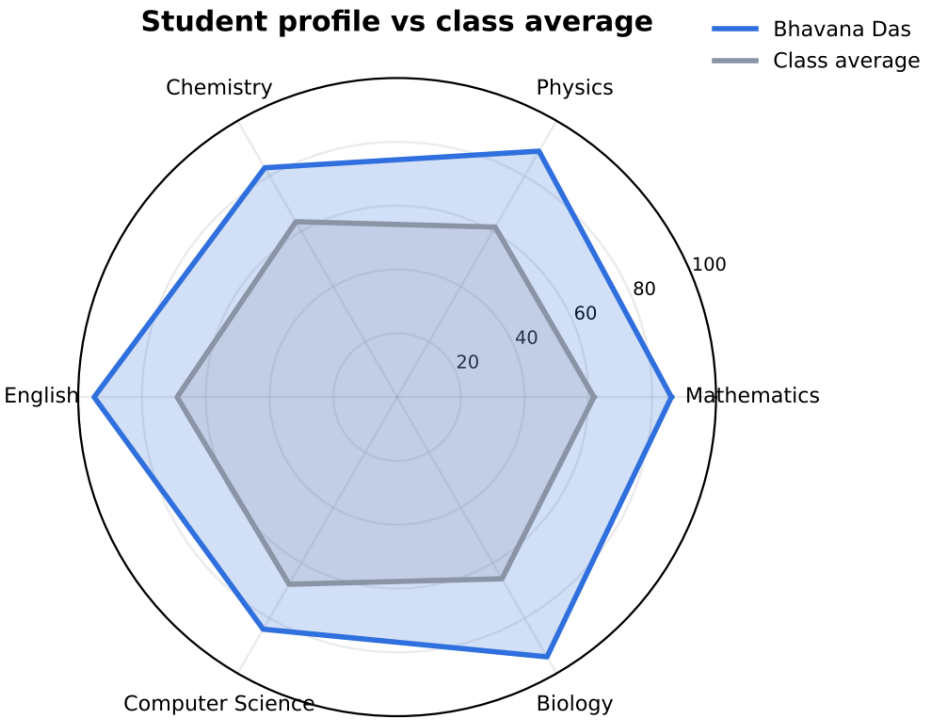


Figure 10: *Student Profile page: radar chart of the top-scoring student against the class average.*

Note: The blue polygon is Bhavana Das; the grey polygon is the class average in each subject. Any student can be chosen from the drop-down list.

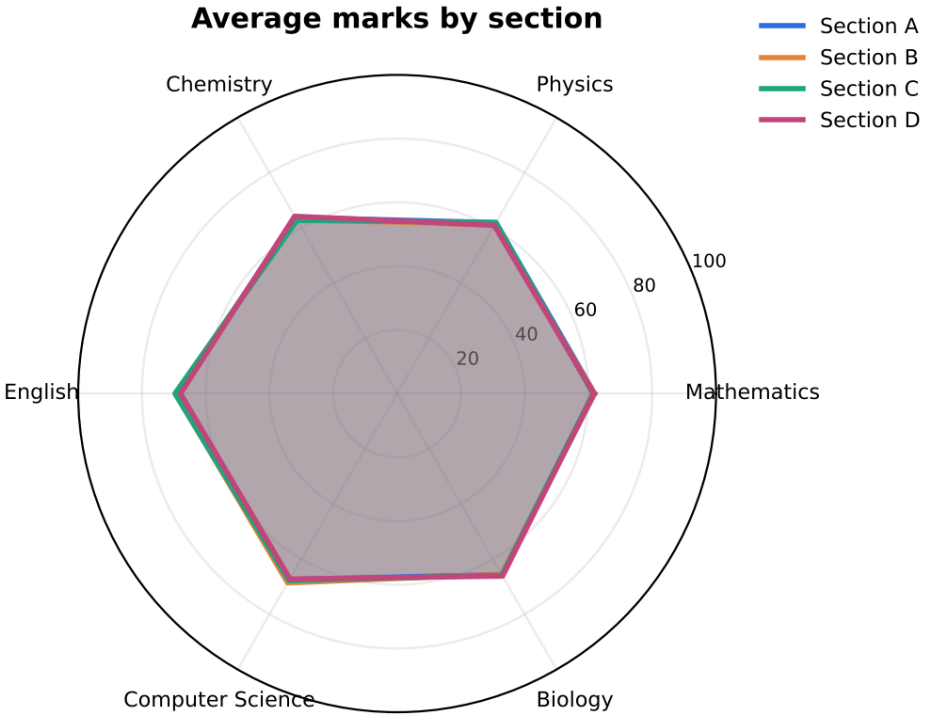


Figure 11: *Compare page: radar chart of the average marks of each section.*

Note: The four section polygons almost coincide, showing that the sections perform alike in every subject.

Correlations page

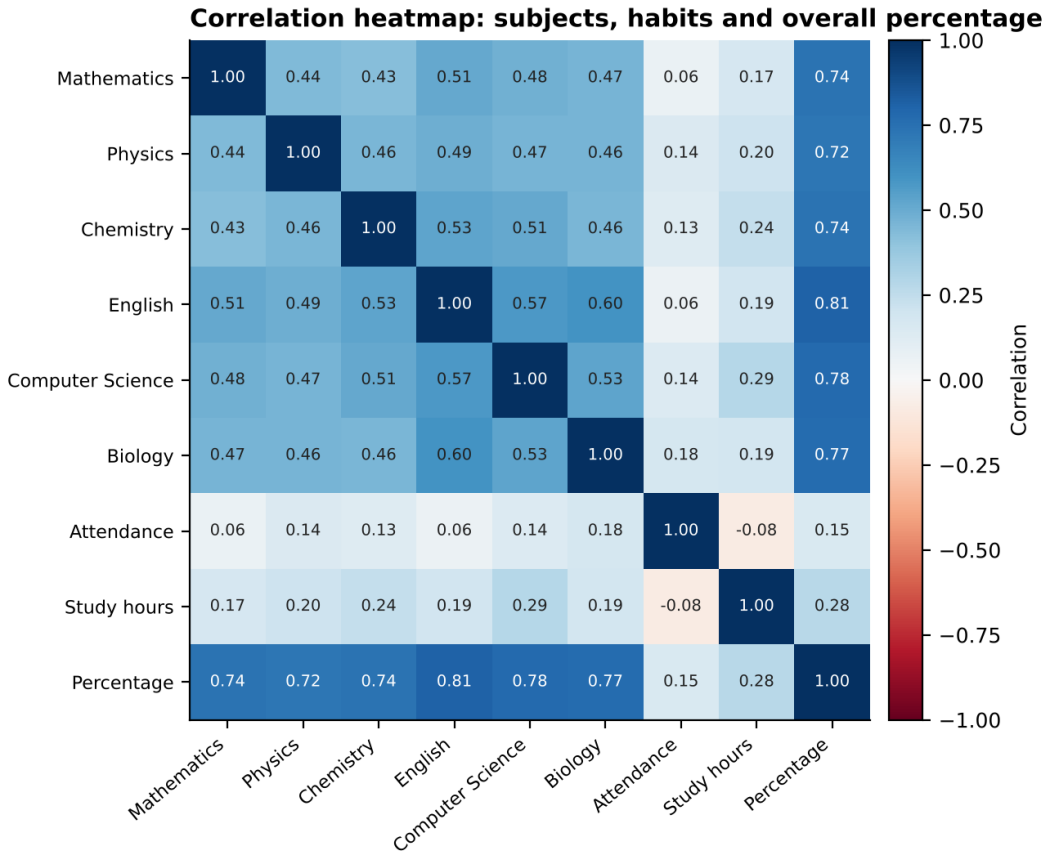


Figure 12: *Correlations page: correlation heatmap of subjects, attendance, study hours and overall percentage.*

Note: All subjects are positively correlated with each other (from 0.43 for Mathematics and Chemistry to 0.60 for English and Biology). Study hours (0.28) and attendance (0.15) have weaker positive links with overall percentage.

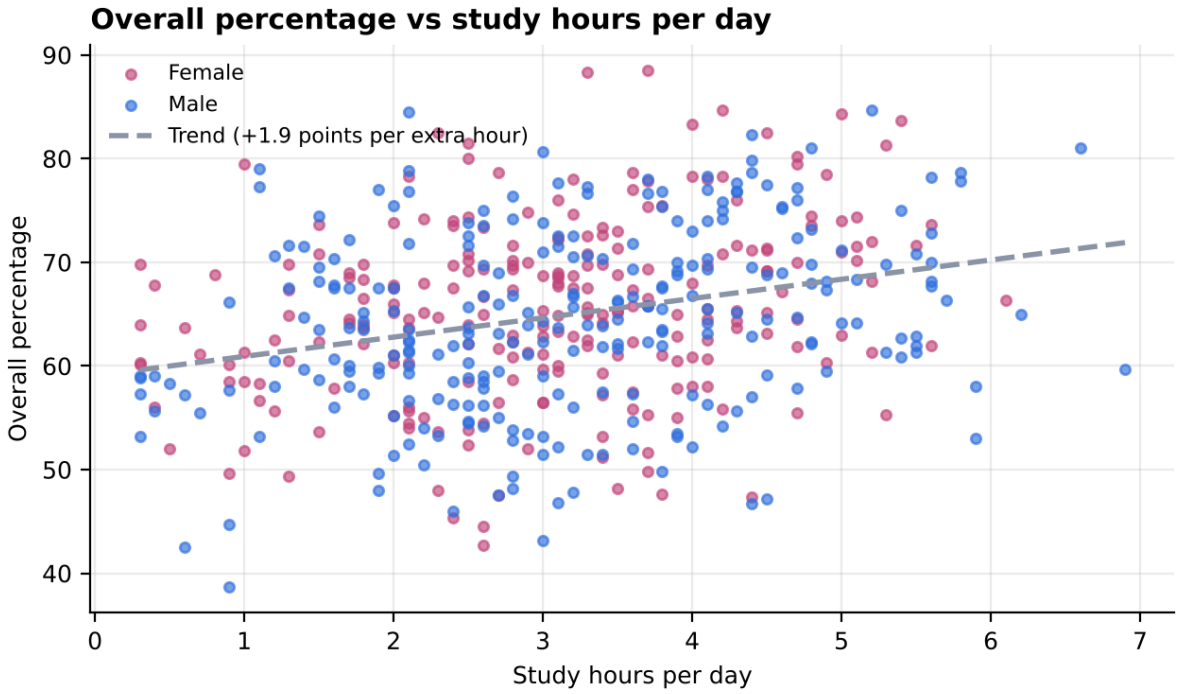


Figure 13: *Correlations page: scatter plot of overall percentage against study hours per day, coloured by gender, with trend line.*

Note: Students who study more tend to score higher: the trend line rises by about 1.9 percentage points for each extra hour per day, but the points are widely scattered.

Data Explorer page

Data Explorer: top 10 students by percentage (sorted view)

ID	Name	Gender	Sec.	Att. %	Maths	Phy.	Chem.	Eng.	CS	Bio.	Pct.	Grade	Result
STU1183	Bhavana Das	Female	B	100.0	86	89	83	95	84	94	88.5	A	Pass
STU1021	Pooja Yadav	Female	C	95.5	96	70	84	100	80	100	88.33	A	Pass
STU1499	Ravi Rao	Male	C	78.7	80	77	86	86	96	83	84.67	A	Pass
STU1130	Mounika Kumar	Female	C	86.0	85	82	74	83	93	91	84.67	A	Pass
STU1001	Harsha Prasad	Male	C	91.6	90	70	79	93	100	75	84.5	A	Pass
STU1090	Sravani Kumar	Female	D	85.1	85	78	87	80	89	87	84.33	A	Pass
STU1016	Bhavana Joshi	Female	A	96.9	91	85	73	83	86	84	83.67	A	Pass
STU1066	Riya Singh	Female	A	75.2	72	80	78	94	90	86	83.33	A	Pass
STU1361	Meera Gupta	Female	D	89.2	82	86	83	85	93	66	82.5	A	Pass
STU1420	Riya Das	Female	C	77.3	77	76	81	97	76	88	82.5	A	Pass

Figure 14: *Data Explorer page: table of students sorted by percentage (top 10 rows shown).*

Note: The explorer lets the user search by name or ID, sort by any column, view summary statistics and download the filtered rows as a CSV file.

9.2 Visualization Results

The current implementation produces the following visual elements. Each chart is drawn with Plotly, so the user can hover over a bar, slice or point for exact values and can zoom and pan.

Visualization	Page	Chart type	Fields used
KPI cards	All pages	Metric cards	Student count, Percentage, pass rate, Attendance_%
Average marks per subject	Overview	Vertical bar + reference line	Six subject columns, pass mark
Overall percentage distribution	Overview	Histogram	Percentage
Grade distribution	Overview	Donut	Grade
Pass and fail by section	Overview	Grouped bar	Section, Result
Top / bottom students	Overview	Horizontal bars	Student_Name, Student_ID, Percentage
Average marks and pass rate	Subject Analysis	Bar + line	Subject marks, pass mark
Marks distribution by subject	Subject Analysis	Box plot	Six subject columns
Single-subject distribution	Subject Analysis	Histogram by gender	Chosen subject, Gender
Group comparison	Subject Analysis	Grouped bar	Section / Gender / Grade / Result, subjects
Section by subject	Subject Analysis	Heatmap	Section, subject marks
Student against class average	Student Profile	Radar + grouped bar	Subject marks of one student, class mean
Group or student comparison	Compare	Radar	Group means or marks of 2–6 students
Variable correlations	Correlations	Heatmap	Subjects, Attendance_%, Study_Hours_Per_Day, Percentage
Relationship explorer	Correlations	Scatter + trend line	Any two numeric fields, colour by Gender/Section/Result/Grade
Student table and download	Data Explorer	Table + download	All columns

Interactive filters. The sidebar filters (section, gender, result, percentage range, attendance range) and the pass mark slider change every KPI, chart and table. Individual pages add their own controls: the number of top and bottom students, the subject, the grouping, the student or students to show, the scatter axes and colour, and the search, sort order and number of rows in the Data Explorer.

9.3 Analysis of Results

Overall performance. The 500 students have an average percentage of 64.9% (median 64.8%, standard deviation 8.7). With a pass mark of 35 in every subject, 483 students pass and 17 fail, a pass rate of 96.6%. The highest percentage is 88.5% (Bhavana Das, Section B) and the lowest 38.7% (Kiran Singh, Section A). Most students fall in grade B (217 students, 43.4%) or B+ (121, 24.2%); 18 students (3.6%) reach grade A and none reaches A+.

Subjects. Subject averages, from highest to lowest, are: English 69.0, Computer Science 67.7, Biology 65.8, Chemistry 63.5, Mathematics 61.7, Physics 61.5. The difference between the highest and lowest subject average is 7.4 marks. The 18 marks below 35 that cause the 17 failures are concentrated in Mathematics (7), with Chemistry (3), Biology (3), Physics (2), Computer Science (2) and English (1) following; one student fails more than one subject. The box plots show low outliers in several subjects, which correspond to the unusually low marks that were deliberately included in the data.

Sections and gender. The four sections have almost identical average percentages — A 64.9%, B 64.9%, C 64.9%, D 64.7% — and a similar profile in every subject, so section does not separate strong from weak performance in this data. Female students (238) average 65.5% and male students (262) 64.3%, a difference of 1.2 points; pass rates are 97.1% and 96.2%.

Study hours and attendance. Study hours per day have a correlation of 0.28 with overall percentage and attendance 0.15, both positive but modest. Grouped by study time, students who study under 2 hours a day average 62.1% (92 students), those who study 2–4 hours 63.9% (271) and those who study over 4 hours 68.6% (137). Grouped by attendance, students below 75% average 62.7% (80 students), those at 75–90% 64.9% (302) and those above 90% 66.3% (118).

Relationships between subjects. Every pair of subjects is positively correlated. The strongest pair is English and Biology (0.60) and the weakest is Mathematics and Chemistry (0.43). Each subject correlates between 0.72 and 0.81 with overall percentage, partly because each subject contributes one-sixth of the percentage itself.

Caution. The dataset is synthetic. The relationships above reflect the assumptions used to generate the data, and correlation does not show that one factor causes another. The numbers illustrate how the dashboard works and should not be read as findings about real students.

9.4 Key Insights

- The dataset covers 500 students: 483 pass (96.6%) and 17 fail at a pass mark of 35 in each subject.
- The average percentage is 64.9%, and most students fall in grades B and B+ (43.4% and 24.2%).
- English is the strongest subject (average 69.0) and Physics the weakest (61.5); Mathematics accounts for the most marks below the pass mark (7 of 18).
- The four sections perform almost identically (averages 64.7% to 64.9%), and the gender gap is only 1.2 points.
- Students who study more than 4 hours a day average 68.6%, compared with 62.1% for those who study under 2 hours.
- Attendance has a weaker link with percentage (correlation 0.15) than study hours (0.28).
- Percentage-based charts hide individual weaknesses: a student can pass overall yet have one very low subject, which is why the Student Profile radar chart is useful.
- All results come from a synthetic dataset and show how the dashboard works, not conclusions about real students.

10. ADVANTAGES AND LIMITATIONS

10.1 Advantages

- Combines marks, attendance and study habits in one interface with six task-oriented pages.
- Interactive filters and charts make comparisons between subjects, sections and students quick.
- The pass mark can be changed and all results, grades and charts update immediately.
- Input validation and clear messages prevent crashes on missing or invalid data.
- Built only with free Python tools and deployed online, so it can be opened without installing anything.

10.2 Limitations

- The dataset is synthetic and small; findings cannot be generalised to real students.
- Only six subjects, one term and two habit measures are included; there is no time dimension.
- The dashboard describes patterns; it does not predict results or prove causes.
- There is no login, so it is not suitable for confidential real student records as it stands.

11. CONCLUSION

The Student Marks Analytics Dashboard shows how a small Python application can turn a table of 500 student records into an interactive tool for exploring performance. By combining KPI cards, filters and a range of Plotly charts — bar, histogram, donut, box plot, radar, heatmap and scatter — it answers questions about subjects, sections, individual students and study habits in seconds. The project demonstrates the complete visualization workflow of data preparation, validation, chart design, interaction and deployment, and its conclusions remain limited to what the synthetic dataset and the implemented features support.

12. FUTURE ENHANCEMENT

- Use real, anonymised student data collected with consent, with the same validation checks.
- Add several terms or exams so that progress over time can be plotted.
- Add more subjects and internal/external marks, and a configurable grading scheme.
- Add login and role-based views for teachers and students.
- Add simple prediction or early-warning indicators for students at risk, with a clear explanation of their limits.
- Add PDF export of the filtered view and a downloadable data dictionary.

13. REFERENCES

- Live dashboard: https://assignment-2-project-2hs4f5lzywtf2sdwzvk79t.streamlit.app/
- Streamlit Documentation: https://docs.streamlit.io
- Pandas Documentation: https://pandas.pydata.org/docs/
- NumPy Documentation: https://numpy.org/doc/
- Plotly Python Documentation: https://plotly.com/python/
- Streamlit Community Cloud Documentation: https://docs.streamlit.io/deploy/streamlit-community-cloud

14. APPENDIX

A. Source Code

Live dashboard: https://assignment-2-project-2hs4f5lzywtf2sdwzvk79t.streamlit.app/

The complete dashboard is written as a single Streamlit file, app.py, which sits next to the data folder. The project folder is organised as follows:

File	Purpose
app.py	Streamlit application: data loading, validation, filters, charts and page layout
requirements.txt	Python libraries required
README.md	Run instructions and page descriptions
data/student_marks_dataset.csv	Student marks dataset (500 students)

Libraries used: streamlit, pandas, numpy and plotly (plotly.express and plotly.graph_objects), plus the Python standard-library modules io and pathlib.

To run: pip install -r requirements.txt, then run `python -m streamlit run app.py` from the project folder. The dashboard opens at http://localhost:8501.

Note: the code contains exactly the six pages offered in the sidebar menu: Overview, Subject Analysis, Student Profile, Compare, Correlations and Data Explorer. Emoji characters used in page names and titles are omitted from the listing below because the print font cannot show them; very long lines are wrapped.

Complete source code (app.py)

```
"""
 Student Marks Analytics Dashboard
==================================
Interactive visualization of student marks (percentages) across subjects.

Run with:  streamlit run app.py

Works with any CSV that has Student_ID, Student_Name, Gender, Section and one
numeric column per subject (marks out of 100). Extra columns such as
Attendance_% and Study_Hours_Per_Day are used when present.
"""

from __future__ import annotations

import io
from pathlib import Path

import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
import streamlit as st

st.set_page_config(
    page_title="Student Marks Analytics Dashboard",
    page_icon="",
    layout="wide",
    initial_sidebar_state="expanded",
)

# ─────────────────────────────────────────────────────────────────────────────
# Constants
# ─────────────────────────────────────────────────────────────────────────────
APP_DIR = Path(__file__).resolve().parent
DATA_PATH = APP_DIR / "data" / "student_marks_dataset.csv"

ID, NAME, GENDER, SECTION = "Student_ID", "Student_Name", "Gender", "Section"
ATT, STUDY = "Attendance_%", "Study_Hours_Per_Day"
REQUIRED = [ID, NAME, GENDER, SECTION]
DERIVED = ["Total_Marks", "Percentage", "Subjects_Failed", "Result", "Grade"]
NON_SUBJECT = set(REQUIRED + [ATT, STUDY] + DERIVED)

GRADE_ORDER = ["A+", "A", "B+", "B", "C", "D", "F"]
COLORWAY = ["#2F6FDE", "#E8833A", "#1FA67A", "#C2457B", "#7A5CD6", "#D4A017", "#2AA3B8", "#8A94A6"]
GOOD, BAD, NEUTRAL = "#1FA67A", "#D9534F", "#9AA4B2"
px.defaults.color_discrete_sequence = COLORWAY

PAGES = [
    " Overview",
    " Subject Analysis",
    " Student Profile",
    " Compare",
    " Correlations",
    " Data Explorer",
]

try:
    _V = tuple(int(p) for p in st.__version__.split(".")[:2])
except Exception:  # pragma: no cover
    _V = (1, 40)
NEW_WIDTH_API = _V >= (1, 50)  # Streamlit ≥ 1.50 replaced use_container_width


# ─────────────────────────────────────────────────────────────────────────────
# Styling and small helpers
# ─────────────────────────────────────────────────────────────────────────────
def inject_css() -> None:
    st.markdown(
        """
        <style>
        .block-container { padding-top: 1.6rem; padding-bottom: 3rem; max-width: 1500px; }
        h1 { font-weight: 650; letter-spacing: -0.02em; margin-bottom: 0.1rem; }
        h2, h3 { font-weight: 600; letter-spacing: -0.01em; }
        .app-subtitle { font-size: 1.02rem; opacity: 0.72; margin: 0 0 1.1rem 0; }
        div[data-testid="stMetric"] {
            background: rgba(128,128,128,0.07); border: 1px solid rgba(128,128,128,0.22);
            border-left: 3px solid #2F6FDE; border-radius: 8px; padding: 0.7rem 0.9rem;
        }
        div[data-testid="stMetricValue"] { font-variant-numeric: tabular-nums; font-weight: 600; }
        .insight { border-left: 3px solid #1FA67A; padding: 0.35rem 0.8rem; margin: 0.2rem 0 0.8rem 0;
                   background: rgba(31,166,122,0.08); border-radius: 0 6px 6px 0; font-size: 0.93rem; }
        .footer-note { font-size: 0.85rem; opacity: 0.7; }
        </style>
        """,
        unsafe_allow_html=True,
    )


def insight(text: str) -> None:
    st.markdown(f'<div class="insight">{text}</div>', unsafe_allow_html=True)
```

```
def ok(x) -> bool:
    try:
        return x is not None and np.isfinite(float(x))
    except (TypeError, ValueError):
        return False


def pct(x, digits: int = 1) -> str:
    return f"{float(x):.{digits}f}%" if ok(x) else "—"


PLOT_CONFIG = {"displaylogo": False, "scrollZoom": True, "modeBarButtonsToRemove": ["lasso2d", "select2d"]}


def show(fig: go.Figure, key: str | None = None) -> None:
    if NEW_WIDTH_API:
        st.plotly_chart(fig, width="stretch", config=PLOT_CONFIG, key=key)
    else:
        st.plotly_chart(fig, use_container_width=True, config=PLOT_CONFIG, key=key)


def show_table(df, height: int | None = None, **kwargs) -> None:
    kwargs.setdefault("hide_index", True)
    if height is not None:
        kwargs["height"] = height
    if NEW_WIDTH_API:
        st.dataframe(df, width="stretch", **kwargs)
    else:
        st.dataframe(df, use_container_width=True, **kwargs)


def style(fig: go.Figure, title: str, x: str | None = None, y: str | None = None,
          height: int = 400, legend: bool = True) -> go.Figure:
    fig.update_layout(
        title=dict(text=title, x=0.0, xanchor="left", font=dict(size=16)),
        height=height, margin=dict(l=10, r=10, t=56, b=10), showlegend=legend,
        legend=dict(orientation="h", yanchor="bottom", y=1.0, xanchor="right", x=1.0),
    )
    if x is not None:
        fig.update_xaxes(title_text=x)
    if y is not None:
        fig.update_yaxes(title_text=y)
    return fig


# ──────────────────────────────────────────────────────────────────────────
# Data loading and preparation
# ──────────────────────────────────────────────────────────────────────────
def prepare(raw: pd.DataFrame) -> tuple[pd.DataFrame, list[str]]:
    """Validate columns, clean types and add Total_Marks / Percentage. Returns (df, subjects)."""
    df = raw.copy()
    df.columns = [str(c).strip() for c in df.columns]
    missing = [c for c in REQUIRED if c not in df.columns]
    if missing:
        raise ValueError(
            "The dataset is missing required column(s): " + ", ".join(missing)
            + ". Expected at least: Student_ID, Student_Name, Gender, Section and one column per subject."
        )

    candidates = [c for c in df.columns if c not in NON_SUBJECT]
    subjects = []
    for c in candidates:
        num = pd.to_numeric(df[c], errors="coerce")
        if num.notna().mean() >= 0.8:  # mostly numeric → a subject column
            df[c] = num.clip(0, 100)
            subjects.append(c)
    if len(subjects) < 2:
        raise ValueError("Could not find at least two numeric subject columns (marks out of 100).")

    for c in [ID, NAME, GENDER, SECTION]:
        df[c] = df[c].fillna("Unknown").astype(str).str.strip()
    for c in [ATT, STUDY]:
        if c in df.columns:
            df[c] = pd.to_numeric(df[c], errors="coerce")

    df = df.dropna(subset=subjects, how="all").drop_duplicates(subset=[ID], keep="first")
    df["Total_Marks"] = df[subjects].sum(axis=1, skipna=True)
    df["Percentage"] = df[subjects].mean(axis=1, skipna=True).round(2)  # each subject is out of 100
    return df.reset_index(drop=True), subjects


@st.cache_data(show_spinner="Loading dataset…")
def load_path(path: str, mtime: float):
    return prepare(pd.read_csv(path))


@st.cache_data(show_spinner="Reading the uploaded file…")
def load_bytes(raw: bytes):
    return prepare(pd.read_csv(io.BytesIO(raw)))


def get_dataset():
    error, result = None, None
    if DATA_PATH.exists():
```

```
        try:
            result = load_path(str(DATA_PATH), DATA_PATH.stat().st_mtime)
        except ValueError as exc:
            error = str(exc)
        except Exception as exc:
            error = f"Could not read {DATA_PATH.name}: {exc}"
    else:
        st.error(f"Dataset not found. Put **student_marks_dataset.csv** in `{DATA_PATH.parent}` or upload it
            below.")
        up = st.file_uploader("Upload student marks CSV", type="csv")
        if up is None:
            st.stop()
        try:
            result = load_bytes(up.getvalue())
        except ValueError as exc:
            error = str(exc)
        except Exception as exc:
            error = f"Could not read the uploaded file: {exc}"
    if error:
        st.error(error)
        st.stop()
    if result is None or result[0].empty:
        st.error("The dataset has no usable rows after cleaning.")
        st.stop()
    return result


def grade_of(p: float, failed: int) -> str:
    if failed > 0:
        return "F"
    return "A+" if p >= 90 else "A" if p >= 80 else "B+" if p >= 70 else "B" if p >= 60 else "C" if p >= 50 else
         "D"


def derive(df: pd.DataFrame, subjects: list[str], pass_mark: int) -> pd.DataFrame:
    """Result / Subjects_Failed / Grade depend on the pass mark chosen in the sidebar."""
    out = df.copy()
    out["Subjects_Failed"] = (out[subjects] < pass_mark).sum(axis=1)
    out["Result"] = np.where(out["Subjects_Failed"] == 0, "Pass", "Fail")
    out["Grade"] = [grade_of(p, int(n)) for p, n in zip(out["Percentage"], out["Subjects_Failed"])]
    return out


# ─────────────────────────────────────────────────────────────────────────────
# Sidebar
# ─────────────────────────────────────────────────────────────────────────────
def build_sidebar(df: pd.DataFrame, subjects: list[str]):
    sb = st.sidebar
    sb.markdown("###  Student Analytics")
    page = sb.radio("Navigate", PAGES, label_visibility="collapsed", key="nav_page")
    sb.divider()

    with sb.expander("Advanced controls"):
        pass_mark = st.slider("Pass mark (per subject)", 0, 60, 35, key="pass_mark",
                              help="A student fails if any subject is below this mark.")

    d = derive(df, subjects, pass_mark)
    sb.markdown("#### Filters")
    sections = sb.multiselect("Section", sorted(d[SECTION].unique()), key="f_section",
                              placeholder="All sections")
    genders = sb.multiselect("Gender", sorted(d[GENDER].unique()), key="f_gender",
                             placeholder="All genders")
    results = sb.multiselect("Result", ["Pass", "Fail"], key="f_result", placeholder="Pass and Fail")
    lo, hi = int(np.floor(d["Percentage"].min())), int(np.ceil(d["Percentage"].max()))
    pct_rng = sb.slider("Percentage range", lo, hi, (lo, hi), key="f_pct") if lo < hi else (lo, hi)
    att_rng = None
    if ATT in d.columns and d[ATT].notna().any():
        a_lo, a_hi = int(np.floor(d[ATT].min())), int(np.ceil(d[ATT].max()))
        if a_lo < a_hi:
            att_rng = sb.slider("Attendance (%)", a_lo, a_hi, (a_lo, a_hi), key="f_att")

    mask = d["Percentage"].between(pct_rng[0], pct_rng[1])
    if sections:
        mask &= d[SECTION].isin(sections)
    if genders:
        mask &= d[GENDER].isin(genders)
    if results:
        mask &= d["Result"].isin(results)
    if att_rng is not None:
        mask &= d[ATT].between(att_rng[0], att_rng[1]) | d[ATT].isna()
    f = d[mask]

    sb.divider()
    sb.caption(f"**{len(f):,}** of **{len(d):,}** students match · {len(subjects)} subjects · pass mark
        {pass_mark}")
    return page, d, f, pass_mark


# ─────────────────────────────────────────────────────────────────────────────
# KPIs
# ─────────────────────────────────────────────────────────────────────────────
def render_kpis(f: pd.DataFrame, subjects: list[str]) -> None:
    means = f[subjects].mean()
    top = f.loc[f["Percentage"].idxmax()]
```

```
    low = f.loc[f["Percentage"].idxmin()]
    c = st.columns(6)
    c[0].metric("Students", f"{len(f):,}")
    c[1].metric("Average percentage", pct(f["Percentage"].mean()))
    c[2].metric("Highest percentage", pct(top["Percentage"]), help=f"{top[NAME]} ({top[ID]})")
    c[3].metric("Lowest percentage", pct(low["Percentage"]), help=f"{low[NAME]} ({low[ID]})")
    c[4].metric("Pass rate", pct((f["Result"] == "Pass").mean() * 100))
    if ATT in f.columns and f[ATT].notna().any():
        c[5].metric("Average attendance", pct(f[ATT].mean()))
    else:
        c[5].metric("Top subject", str(means.idxmax()), help=f"Average {means.max():.1f}")


# ─────────────────────────────────────────────────────────────────────────────
# Pages
# ─────────────────────────────────────────────────────────────────────────────
def page_overview(f: pd.DataFrame, subjects: list[str], pass_mark: int) -> None:
    means = f[subjects].mean()
    best, worst = means.idxmax(), means.idxmin()
    insight(f"<b>{best}</b> has the highest average ({means[best]:.1f}) and <b>{worst}</b> the lowest
        ({means[worst]:.1f}). "
            f"{(f['Result'] == 'Fail').sum():,} of {len(f):,} students fail at least one subject.")

    c1, c2 = st.columns(2)
    with c1:
        fig = go.Figure(go.Bar(
            x=means.index, y=means.values, marker_color=COLORWAY[0],
            text=[f"{v:.1f}" for v in means.values], textposition="outside",
            hovertemplate="<b>%{x}</b><br>Average: %{y:.2f}<extra></extra>",
        ))
        fig.add_hline(y=pass_mark, line_dash="dash", line_color=BAD, annotation_text=f"pass mark {pass_mark}")
        fig.update_yaxes(range=[0, 100])
        show(style(fig, "Average marks per subject", "Subject", "Average marks", legend=False), key="ov_avg")
    with c2:
        fig = px.histogram(f, x="Percentage", nbins=20)
        fig.update_traces(marker_color=COLORWAY[1], marker_line_width=0.5, marker_line_color="rgba(0,0,0,0.25)",
                          hovertemplate="Percentage: %{x}<br>Students: %{y}<extra></extra>")
        fig.add_vline(x=float(f["Percentage"].mean()), line_dash="dash", line_color=NEUTRAL,
                      annotation_text=f"mean {f['Percentage'].mean():.1f}%")
        show(style(fig, "Distribution of overall percentage", "Overall percentage", "Number of students",
                   legend=False), key="ov_hist")

    c3, c4 = st.columns(2)
    with c3:
        counts = f["Grade"].value_counts().reindex(GRADE_ORDER).dropna()
        colors = [BAD if g == "F" else COLORWAY[i % len(COLORWAY)] for i, g in enumerate(counts.index)]
        fig = go.Figure(go.Pie(labels=counts.index, values=counts.values, hole=0.5, sort=False,
                               marker=dict(colors=colors), textinfo="label+percent",
                               hovertemplate="<b>Grade %{label}</b><br>%{value} students
                                   (%{percent})<extra></extra>"))
        show(style(fig, "Grade distribution", height=400), key="ov_grade")
    with c4:
        g = f.groupby([SECTION, "Result"]).size().reset_index(name="Students")
        fig = px.bar(g, x=SECTION, y="Students", color="Result", barmode="group",
                     color_discrete_map={"Pass": GOOD, "Fail": BAD})
        show(style(fig, "Pass and fail by section", "Section", "Students"), key="ov_section")

    st.markdown("### Top and bottom performers")
    n = st.radio("List length", [5, 10, 15], index=1, horizontal=True, key="ov_n", format_func=lambda x: f"Top
        {x}")
    cols = [ID, NAME, SECTION, "Percentage", "Grade"]
    l, r = st.columns(2)
    with l:
        top = f.nlargest(n, "Percentage")[cols]
        fig = go.Figure(go.Bar(x=top["Percentage"], y=top[NAME] + " (" + top[ID] + ")", orientation="h",
                               marker_color=GOOD, hovertemplate="%{y}<br>%{x:.2f}%<extra></extra>"))
        fig.update_yaxes(autorange="reversed")
        show(style(fig, f"Top {n} students", "Percentage", None, height=max(300, 28 * n + 90), legend=False),
            key="ov_top")
    with r:
        bottom = f.nsmallest(n, "Percentage")[cols]
        fig = go.Figure(go.Bar(x=bottom["Percentage"], y=bottom[NAME] + " (" + bottom[ID] + ")",
            orientation="h",
                               marker_color=BAD, hovertemplate="%{y}<br>%{x:.2f}%<extra></extra>"))
        fig.update_yaxes(autorange="reversed")
        show(style(fig, f"Bottom {n} students — may need support", "Percentage", None,
                   height=max(300, 28 * n + 90), legend=False), key="ov_bottom")


def page_subjects(f: pd.DataFrame, subjects: list[str], pass_mark: int) -> None:
    st.subheader(" Subject analysis")
    t1, t2, t3, t4, t5 = st.tabs(["Averages and pass rate", "Distribution", "Single subject", "By group",
        "Section heatmap"])

    with t1:
        stats = pd.DataFrame({
            "Subject": subjects,
            "Average": [f[s].mean() for s in subjects],
            "Median": [f[s].median() for s in subjects],
            "Highest": [f[s].max() for s in subjects],
            "Lowest": [f[s].min() for s in subjects],
            "Std deviation": [f[s].std() for s in subjects],
            "Pass rate %": [(f[s] >= pass_mark).mean() * 100 for s in subjects],
        })
```

```
        fig = go.Figure()
        fig.add_trace(go.Bar(x=stats["Subject"], y=stats["Average"], name="Average", marker_color=COLORWAY[0]))
        fig.add_trace(go.Scatter(x=stats["Subject"], y=stats["Pass rate %"], name="Pass rate %",
            mode="lines+markers",
                                 line=dict(color=COLORWAY[1], width=2.4)))
        fig.update_yaxes(range=[0, 105])
        show(style(fig, "Average marks and pass rate by subject", "Subject", "Marks / pass rate (%)"),
            key="sa_avg")
        show_table(stats.style.format({c: "{:.1f}" for c in stats.columns if c != "Subject"}))
        hardest = stats.loc[stats["Pass rate %"].idxmin()]
        insight(f"<b>{hardest['Subject']}</b> has the lowest pass rate ({hardest['Pass rate %']:.1f}%).")

    with t2:
        fig = go.Figure()
        for i, s in enumerate(subjects):
            fig.add_trace(go.Box(y=f[s], name=s, boxpoints="outliers", marker_color=COLORWAY[i %
                len(COLORWAY)]))
        fig.add_hline(y=pass_mark, line_dash="dash", line_color=BAD)
        show(style(fig, "Marks distribution by subject (box plot — dots are outliers)", "Subject", "Marks",
                   height=480, legend=False), key="sa_box")

    with t3:
        subj = st.selectbox("Subject", subjects, key="sa_subject")
        v = f[subj].dropna()
        m = st.columns(4)
        m[0].metric("Average", f"{v.mean():.1f}")
        m[1].metric("Highest", f"{v.max():.0f}")
        m[2].metric("Lowest", f"{v.min():.0f}")
        m[3].metric("Pass rate", pct((v >= pass_mark).mean() * 100))
        fig = px.histogram(f, x=subj, nbins=20, color=GENDER, barmode="overlay", opacity=0.7)
        fig.add_vline(x=pass_mark, line_dash="dash", line_color=BAD, annotation_text="pass mark")
        show(style(fig, f"{subj} — marks distribution by gender", "Marks", "Number of students"),
            key="sa_single")
        top = f.nlargest(5, subj)[[ID, NAME, SECTION, subj]]
        st.markdown(f"**Top 5 in {subj}**")
        show_table(top)

    with t4:
        grp = st.radio("Group by", [c for c in [SECTION, GENDER, "Grade", "Result"] if c in f.columns],
                       horizontal=True, key="sa_group")
        means = f.groupby(grp)[subjects].mean().reset_index().melt(id_vars=grp, var_name="Subject",
            value_name="Average")
        fig = px.bar(means, x="Subject", y="Average", color=grp, barmode="group")
        fig.update_yaxes(range=[0, 100])
        show(style(fig, f"Average marks by subject and {grp.lower()}", "Subject", "Average marks"),
            key="sa_group_fig")

    with t5:
        piv = f.groupby(SECTION)[subjects].mean()
        fig = go.Figure(go.Heatmap(
            z=piv.values, x=piv.columns, y=piv.index, colorscale="RdYlGn", text=np.round(piv.values, 1),
            texttemplate="%{text}", hovertemplate="Section %{y} · %{x}<br>Average: %{z:.1f}<extra></extra>",
            colorbar=dict(title=dict(text="Avg")),
        ))
        fig.update_yaxes(autorange="reversed", type="category")
        show(style(fig, "Average marks — section × subject", "Subject", "Section", height=360, legend=False),
             key="sa_heat")


def radar(traces: list[tuple[str, list[float], str]], labels: list[str], title: str) -> go.Figure:
    fig = go.Figure()
    for name, vals, color in traces:
        fig.add_trace(go.Scatterpolar(
            r=list(vals) + [vals[0]], theta=labels + [labels[0]], fill="toself", name=name, opacity=0.55,
            line=dict(color=color, width=2),
            hovertemplate="%{theta}: %{r:.1f}<extra>" + name + "</extra>",
        ))
    fig.update_layout(polar=dict(radialaxis=dict(visible=True, range=[0, 100])))
    return style(fig, title, height=520)


def page_profile(f: pd.DataFrame, subjects: list[str]) -> None:
    st.subheader(" Student profile")
    people = f.sort_values(NAME)
    labels = (people[NAME] + " — " + people[ID]).tolist()
    pick = st.selectbox("Student", labels, key=f"sp_pick_{len(labels)}_{hash(tuple(labels[:5]))}",
                        help="Type a name or ID to search.")
    row = people.iloc[labels.index(pick)]
    rank = int((f["Percentage"] > row["Percentage"]).sum()) + 1

    c = st.columns(5)
    c[0].metric("Percentage", pct(row["Percentage"], 2))
    c[1].metric("Rank", f"{rank} of {len(f)}", help="Within the students matching the current filters.")
    c[2].metric("Grade", str(row["Grade"]))
    c[3].metric("Result", str(row["Result"]))
    if ATT in f.columns and ok(row.get(ATT)):
        c[4].metric("Attendance", pct(row[ATT]))
    else:
        c[4].metric("Section", str(row[SECTION]))

    marks = row[subjects].astype(float)
    avg = f[subjects].mean()
    best, weak = marks.idxmax(), marks.idxmin()
    insight(f"<b>{row[NAME]}</b> is strongest in <b>{best}</b> ({marks[best]:.0f}) and weakest in <b>{weak}</b>
```

```
        ({marks[weak]:.0f}). "
            f"Overall they are {abs(row['Percentage'] - f['Percentage'].mean()):.1f} points "
            f"{'above' if row['Percentage'] >= f['Percentage'].mean() else 'below'} the average of the current
                selection.")

    l, r = st.columns(2)
    with l:
        show(radar([(row[NAME], marks.tolist(), COLORWAY[0]), ("Class average", avg.tolist(), NEUTRAL)],
                   subjects, "Subject profile vs class average"), key="sp_radar")
    with r:
        fig = go.Figure()
        fig.add_trace(go.Bar(x=subjects, y=marks.values, name=row[NAME], marker_color=COLORWAY[0]))
        fig.add_trace(go.Bar(x=subjects, y=avg.values, name="Class average", marker_color=NEUTRAL))
        fig.update_layout(barmode="group")
        fig.update_yaxes(range=[0, 100])
        show(style(fig, "Marks by subject", "Subject", "Marks", height=520), key="sp_bar")


def page_compare(f: pd.DataFrame, subjects: list[str]) -> None:
    st.subheader(" Compare")
    t1, t2 = st.tabs(["Compare groups", "Compare students"])

    with t1:
        grp = st.radio("Compare by", [SECTION, GENDER, "Grade", "Result"], horizontal=True, key="cmp_group")
        means = f.groupby(grp)[subjects].mean()
        if len(means) < 2:
            st.warning(f"Only one {grp.lower()} in the current filter. Widen the filters to compare groups.")
        else:
            traces = [(f"{grp} {k}", means.loc[k].tolist(), COLORWAY[i % len(COLORWAY)]) for i, k in
                enumerate(means.index[:8])]
            show(radar(traces, subjects, f"Average marks by {grp.lower()}"), key="cmp_group_radar")
            show_table(means.round(1).reset_index())

    with t2:
        people = f.sort_values(NAME)
        labels = (people[NAME] + " — " + people[ID]).tolist()
        chosen = st.multiselect("Students (2–6)", labels, default=labels[:3], max_selections=6,
                                key=f"cmp_students_{len(labels)}_{hash(tuple(labels[:5]))}")
        if len(chosen) < 2:
            st.warning("Select at least **two** students to compare.")
        else:
            rows = people.iloc[[labels.index(c) for c in chosen]]
            traces = [(r[NAME], r[subjects].astype(float).tolist(), COLORWAY[i % len(COLORWAY)])
                      for i, (_, r) in enumerate(rows.iterrows())]
            show(radar(traces, subjects, "Student comparison radar"), key="cmp_students_radar")
            show_table(rows[[ID, NAME, SECTION] + subjects + ["Percentage", "Grade"]])


def page_correlations(f: pd.DataFrame, subjects: list[str]) -> None:
    st.subheader(" Correlations")
    numeric = subjects + [c for c in [ATT, STUDY] if c in f.columns and f[c].notna().any()] + ["Percentage"]
    if len(f) < 3:
        st.warning("Need at least 3 students to compute correlations. Widen the filters.")
        return

    corr = f[numeric].corr()
    fig = go.Figure(go.Heatmap(
        z=corr.values, x=corr.columns, y=corr.index, colorscale="RdBu", zmin=-1, zmax=1,
        text=np.round(corr.values, 2), texttemplate="%{text}",
        hovertemplate="%{y} vs %{x}<br>Correlation: %{z:.3f}<extra></extra>",
            colorbar=dict(title=dict(text="Corr.")),
    ))
    fig.update_yaxes(autorange="reversed")
    fig.update_xaxes(tickangle=-35)
    show(style(fig, "Correlation heatmap — subjects, habits and overall percentage", height=560, legend=False),
         key="co_heat")
    if "Percentage" in corr.columns:
        pc = corr["Percentage"].drop("Percentage").dropna()
        if len(pc):
            insight(f"<b>{pc.idxmax()}</b> is most strongly linked to overall percentage (r = {pc.max():.2f}). "
                    "Correlation shows association, not cause.")

    st.markdown("### Scatter explorer")
    c1, c2, c3 = st.columns(3)
    default_x = STUDY if STUDY in numeric else numeric[0]
    xcol = c1.selectbox("X-axis", numeric, index=numeric.index(default_x), key="co_x")
    ycol = c2.selectbox("Y-axis", numeric, index=numeric.index("Percentage"), key="co_y")
    color = c3.selectbox("Colour by", [GENDER, SECTION, "Result", "Grade"], key="co_color")
    plot = f.dropna(subset=[xcol, ycol])
    if plot.empty:
        st.warning("No rows with valid values for these axes.")
        return
    fig = px.scatter(plot, x=xcol, y=ycol, color=color, opacity=0.75, custom_data=[NAME, ID, SECTION,
        "Percentage"])
    fig.update_traces(marker=dict(size=8), hovertemplate="<b>%{customdata[0]}</b> (%{customdata[1]})<br>Section
        %{customdata[2]}"
                      f"<br>{xcol}: " "%{x}" f"<br>{ycol}: " "%{y}<br>Overall:
                          %{customdata[3]:.1f}%<extra></extra>")
    if xcol != ycol and plot[xcol].nunique() > 2:  # simple least-squares trend line
        slope, intercept = np.polyfit(plot[xcol].astype(float), plot[ycol].astype(float), 1)
        xs = np.array([plot[xcol].min(), plot[xcol].max()], dtype=float)
        fig.add_trace(go.Scatter(x=xs, y=slope * xs + intercept, mode="lines", name="Trend",
                                 line=dict(color=NEUTRAL, dash="dash", width=2)))
    show(style(fig, f"{ycol} vs {xcol}", xcol, ycol, height=500), key="co_scatter")
```

```
def page_explorer(f: pd.DataFrame, subjects: list[str]) -> None:
    st.subheader(" Data Explorer")
    c1, c2, c3, c4 = st.columns([3, 2, 1, 1])
    search = c1.text_input("Search by name or ID", placeholder="e.g. Priya or STU1042", key="ex_search").strip()
    sort_col = c2.selectbox("Sort by", list(f.columns), index=list(f.columns).index("Percentage"),
        key="ex_sort")
    order = c3.radio("Order", ["Descending", "Ascending"], key="ex_order")
    n_rows = c4.selectbox("Rows", [50, 100, 250, 500, 1000], index=3, key="ex_rows")

    view = f
    if search:
        hit = view[NAME].str.contains(search, case=False, regex=False) | view[ID].str.contains(search,
            case=False, regex=False)
        view = view[hit]
    if view.empty:
        st.warning("No students match your search and filters.")
        return
    shown = view.sort_values(sort_col, ascending=(order == "Ascending"), kind="stable").head(int(n_rows))
    st.caption(f"Showing **{len(shown):,}** of **{len(view):,}** matching students.")
    show_table(shown, height=520)

    with st.expander("Summary statistics"):
        show_table(view[subjects + ["Total_Marks", "Percentage"]].describe().T.round(2).reset_index()
                   .rename(columns={"index": "column"}))

    st.download_button(" Download Filtered Data", data=view.to_csv(index=False).encode("utf-8"),
                       file_name="filtered_student_marks.csv", mime="text/csv", type="primary")


# ─────────────────────────────────────────────────────────────────────────────
# Main
# ─────────────────────────────────────────────────────────────────────────────
def main() -> None:
    inject_css()
    st.title(" Student Marks Analytics Dashboard")
    st.markdown('<p class="app-subtitle">Interactive analysis of student performance across subjects, sections
        and study habits</p>',
                unsafe_allow_html=True)

    df, subjects = get_dataset()
    page, d, f, pass_mark = build_sidebar(df, subjects)
    if f.empty:
        st.warning("No students match the current filters. Clear a filter or widen the percentage range.")
        st.stop()

    render_kpis(f, subjects)
    st.markdown("")

    if page == PAGES[0]:
        page_overview(f, subjects, pass_mark)
    elif page == PAGES[1]:
        page_subjects(f, subjects, pass_mark)
    elif page == PAGES[2]:
        page_profile(f, subjects)
    elif page == PAGES[3]:
        page_compare(f, subjects)
    elif page == PAGES[4]:
        page_correlations(f, subjects)
    else:
        page_explorer(f, subjects)

    st.divider()
    st.markdown('<p class="footer-note"><em>This dashboard is for educational and analytical purposes
        only.</em></p>',
                unsafe_allow_html=True)


main()
```

B. Dataset Notes

Item	Detail
File	student_marks_dataset.csv (data folder)
Origin	Synthetic data generated with a Python script (NumPy and Pandas) for this project; not real student records
Rows and columns	500 students, 18 columns
Subjects	Mathematics, Physics, Chemistry, English, Computer Science, Biology (marks out of 100)
Other fields	Attendance_% (56.5–100.0), Study_Hours_Per_Day (0.3–6.9), Gender, Section (A–D)
Calculated fields	Total_Marks, Percentage, Subjects_Failed, Result, Grade (recalculated by the app for the chosen pass mark)
Cleaning in the app	Type conversion, range limit 0–100, missing text set to Unknown, duplicate IDs removed
Known limitations	Synthetic and small; relationships reflect the generating assumptions; single term; no real-world validity

C. Live Deployment

The running dashboard can be opened in a web browser at: https://assignment-2-project-2hs4f5lzywtf2sdwzvk79t.streamlit.app/. The figures in Section 9.1 correspond to its pages and charts.